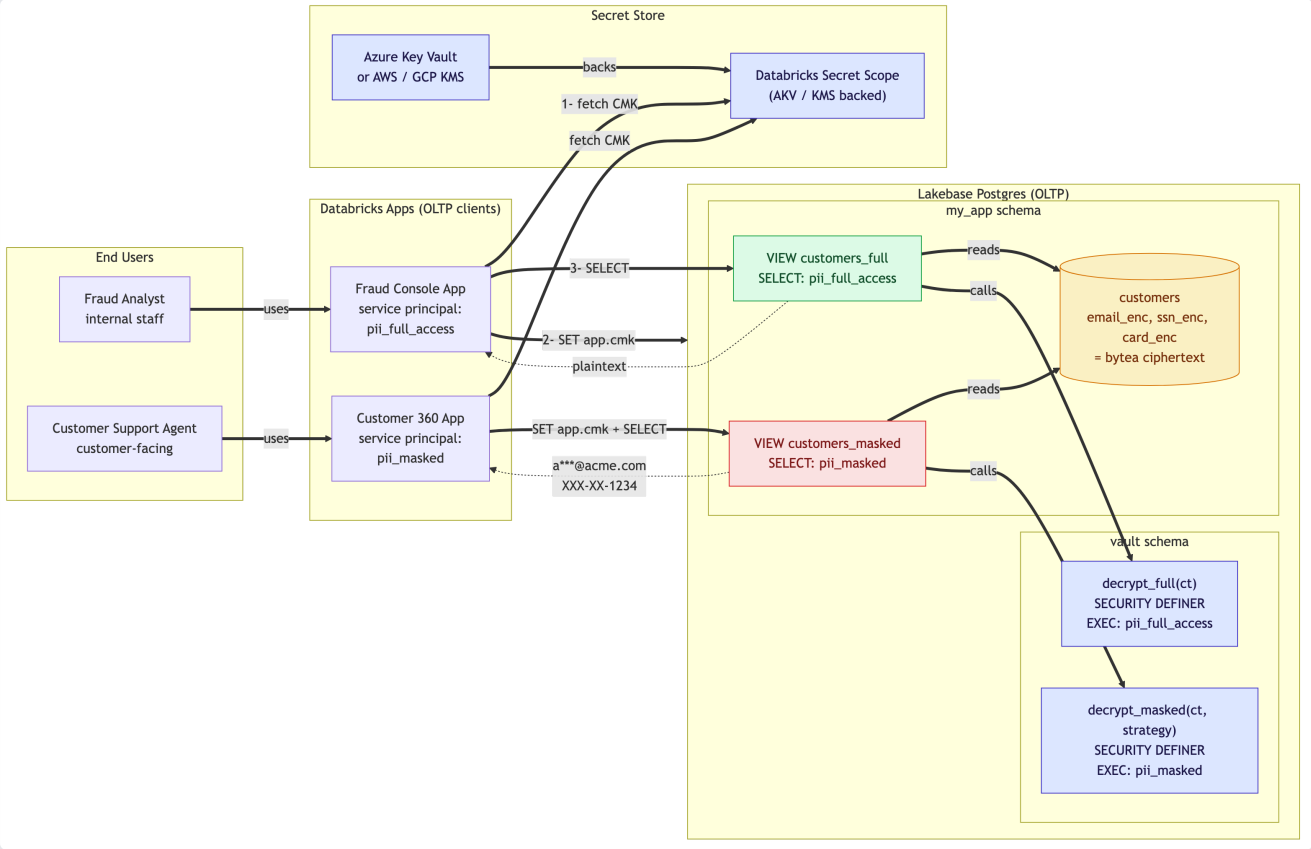
DATABRICKS LAKEBASE · REFERENCE PATTERN

Field-Level Encryption & Dynamic Masking

Use case: Protect PII in OLTP Postgres
Components: Lakebase · pgcrypto AKV / KMS · Databricks Apps

> Encryption-at-rest only protects you against a stolen disk. Once a user authenticates, they see plaintext. **Field-level encryption** stores PII as ciphertext in the table and decrypts only inside an authorized function call, so a **customer-managed key (CMK) from Azure Key Vault / KMS** controls who sees what. Add **role-based masking** and the same row looks completely different to a fraud analyst vs. a support agent.

Architecture



CMK lives in Azure Key Vault → fetched at session start by the Databricks App → injected into Lakebase as a session GUC → decrypt path is gated per Postgres role

How it works (in order)

1. The **customer-managed key (CMK)** is stored in **Azure Key Vault** (or AWS / GCP KMS) and surfaced to Databricks via an AKV-backed Secret Scope. Every key access is audited.
2. A **Databricks App** reads the CMK once when it opens a Postgres connection, and injects it into the session via `SELECT set_config('app.cmk', ...)`. The CMK lives only in that connection's session memory; it is never persisted in Postgres.

3. PII columns in the base table are stored as **bytea ciphertext**, encrypted with `pgp_sym_encrypt(plaintext, cmk)`.
4. Two **SECURITY DEFINER** functions decrypt: `vault.decrypt_full` returns plaintext, `vault.decrypt_masked` returns an already-masked string. Each is granted to a single Postgres role.
5. Two role-specific **views** wire each role to its decrypt path: `customers_full` for `pii_full_access`, `customers_masked` for `pii_masked`. Apps query the view their role can see.

Step 1 — Enable `pgcrypto` and create schemas

Run once in the Lakebase SQL Editor as the project owner. The only Postgres extension required is `pgcrypto`, which is on Lakebase's allowlist by default.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE SCHEMA IF NOT EXISTS my_vault;   -- holds the decrypt functions
CREATE SCHEMA IF NOT EXISTS my_app;     -- holds the encrypted table + views
```

Step 2 — Create the two decrypt functions

Both functions read the CMK from `current_setting('app.cmk')`, which the application sets per session. Marking them `SECURITY DEFINER` lets them be ACL-gated independently from access to the underlying table.

```
CREATE OR REPLACE FUNCTION my_vault.decrypt_full(p_ct bytea)
RETURNS text
LANGUAGE plpgsql SECURITY DEFINER STABLE AS $$
BEGIN
  RETURN public.pgp_sym_decrypt(p_ct, current_setting('app.cmk'));
END; $$;

CREATE OR REPLACE FUNCTION my_vault.decrypt_masked(p_ct bytea, strategy text)
RETURNS text
LANGUAGE plpgsql SECURITY DEFINER STABLE AS $$
DECLARE v text;
BEGIN
  v := public.pgp_sym_decrypt(p_ct, current_setting('app.cmk'));
  RETURN CASE strategy
    WHEN 'email' THEN regexp_replace(v, '(^.).*(@.*$)', '\1***\2')
    WHEN 'ssn'   THEN 'XXX-XX-' || right(v, 4)
    WHEN 'card'  THEN '************' || right(v, 4)
    ELSE '***REDACTED***'
  END;
END; $$;
```

Step 3 — Create roles and ACLs

Each role can call exactly one decrypt path. Neither role can read the CMK directly because `SECURITY DEFINER` means the function runs with the owner's privileges.

```
CREATE ROLE pii_full_access;
CREATE ROLE pii_masked;

REVOKE EXECUTE ON FUNCTION my_vault.decrypt_full(bytea)         FROM PUBLIC;
REVOKE EXECUTE ON FUNCTION my_vault.decrypt_masked(bytea, text) FROM PUBLIC;

GRANT EXECUTE ON FUNCTION my_vault.decrypt_full(bytea)          TO pii_full_access;
GRANT EXECUTE ON FUNCTION my_vault.decrypt_masked(bytea, text)  TO pii_masked;
```

Step 4 — Create the encrypted base table

PII columns are `bytea`. The plaintext never lands in the column. Insert by encrypting with the session CMK.

```
CREATE TABLE my_app.customers (
  id              bigserial PRIMARY KEY,
  full_name       text,
  email_enc       bytea NOT NULL,
  ssn_enc         bytea NOT NULL,
  card_number_enc bytea NOT NULL
);

-- Insert: encrypt with the session CMK (set by the app at connect time)
INSERT INTO my_app.customers(full_name, email_enc, ssn_enc, card_number_enc)
VALUES (
  'Alice Anderson',
  pgp_sym_encrypt('alice@acme.com',    current_setting('app.cmk')),
  pgp_sym_encrypt('123-45-6789',       current_setting('app.cmk')),
  pgp_sym_encrypt('4111111111111111',  current_setting('app.cmk'))
);
```

Step 5 — Create the role-specific views

Two views, one per role. Each calls exactly one decrypt function so the planner-level permission check is also enforced.

```
CREATE VIEW my_app.customers_full AS
SELECT id, full_name,
       my_vault.decrypt_full(email_enc)        AS email,
       my_vault.decrypt_full(ssn_enc)          AS ssn,
       my_vault.decrypt_full(card_number_enc)  AS card_number
FROM my_app.customers;

CREATE VIEW my_app.customers_masked AS
SELECT id, full_name,
       my_vault.decrypt_masked(email_enc,       'email') AS email,
       my_vault.decrypt_masked(ssn_enc,         'ssn')   AS ssn,
       my_vault.decrypt_masked(card_number_enc, 'card')  AS card_number
FROM my_app.customers;

GRANT SELECT ON my_app.customers_full   TO pii_full_access;
GRANT SELECT ON my_app.customers_masked TO pii_masked;
```

Step 6 — Application bootstrap (Python · Databricks App)

The app fetches the CMK from Azure Key Vault via the Databricks Secret Scope, gets a Lakebase OAuth token, opens a connection, sets the session-local CMK, and queries the appropriate view.

```
import psycopg
from databricks.sdk import WorkspaceClient

w   = WorkspaceClient()
cmk = w.dbutils.secrets.get(scope="lakebase-cmk", key="lakebase-cmk")

ep  = w.postgres.get_endpoint(name="projects/<project>/branches/production/endpoints/primary")
tok = w.postgres.generate_database_credential(endpoint=ep.name).token

with psycopg.connect(
        host=ep.status.hosts.host, dbname="databricks_postgres",
        user=w.current_user.me().user_name, password=tok, sslmode="require") as c, c.cursor() as
cur:

    cur.execute("SELECT set_config('app.cmk', %s, false);", (cmk,))   # session-local
    cur.execute("SET ROLE demo_pii_user;")                            # or demo_masked_user
    cur.execute("SELECT * FROM my_app.customers_full;")
    print(cur.fetchall())
```

Sample output — same row, different roles

Role **pii_full_access** reading `my_app.customers_full`

```
1 | Alice Anderson | alice@acme.com | 123-45-6789 | 4111111111111111
```

Role **pii_masked** reading `my_app.customers_masked`

```
1 | Alice Anderson | a***@acme.com | XXX-XX-6789 | ************1111
```

Requirements: Lakebase Autoscaling (PG 16/17) · `pgcrypto` · Databricks Secret Scope backed by AKV / KMS

Full repo (SQL · Python demo · diagrams): github.com/nan-databricks/lakebase-field-level-encryption

Reference pattern
Field Engineering · Databricks